

Module: Problem 3 - Using Bisection Search to Make the Program Faster

This module provides a single utility function that simulates the evolution of a credit-card balance over a year when a fixed minimum payment is applied each month. It repeatedly applies the payment, accrues interest on the remaining unpaid balance, and advances the month counter until twelve months have been processed. The function is useful for financial calculations such as estimating the remaining balance after a year of payments.

Functions

`calculate(month, balance, minPay, monthlyInterestRate)`

Description:

Iteratively updates a credit-card balance for each month until a full year (12 months) has elapsed. For each iteration it:

1. Subtracts the fixed minimum payment (`minPay`) from the current `balance` to obtain the unpaid portion.
2. Applies the monthly interest (`monthlyInterestRate`) to that unpaid portion.
3. Sets the new balance to the sum of the unpaid portion and the accrued interest.
4. Increments the month counter.

The final balance after completing the 12-month cycle is returned.

Parameters: - `month` (*int*): The starting month index (typically `0`). The loop runs while `month < 12` . - `balance` (*float*): The initial outstanding balance on the account. - `minPay` (*float*): The fixed amount paid each month toward the balance. - `monthlyInterestRate` (*float*): The monthly interest rate expressed as a decimal (e.g., `0.02` for 2% per month).

Returns:

- `balance` (*float*): The balance after processing up to month 12. If the supplied `month` is already `≥ 12` , the original `balance` is returned unchanged.

Examples:

```
# Example 1 - typical usage
final_balance = calculate(
    month=0,
    balance=5000.0,          # initial balance
    minPay=200.0,            # fixed monthly payment
    monthlyInterestRate=0.02 # 2 % monthly interest
)
print(final_balance) # → 4235.14 (approximately)

# Example 2 - starting already at month 12 (no iteration)
final_balance = calculate(
    month=12,
    balance=1500.0,
    minPay=100.0,
    monthlyInterestRate=0.015
```

```
)  
print(final_balance) # → 1500.0
```

Edge Cases / Notes: - If `month` is **greater than or equal to 12** when the function is called, the while-loop body never executes and the function returns the input `balance` unchanged. - A `monthlyInterestRate` of **0** results in a linear reduction of the balance by `minPay` each month. - Supplying a `minPay` larger than the current `balance` can produce a negative unpaid balance, which the function will still process (interest on a negative amount yields a further reduction). Callers should validate inputs if negative balances are undesirable. - The function does **not** perform any input validation; passing non-numeric types will raise a `TypeError` during arithmetic operations.